CUA-WM: Augmenting Frozen Computer-Use Agents with SFT-Based World Modeling

Alexander Geppert
Department of Computer Sciences
University of Wisconsin-Madison
ageppert@wisc.edu

Abstract

World-model reasoning (decomposing agent behavior into state estimation, transition prediction, and action selection) has improved performance across web, GUI, and game environments, but typically requires reinforcement learning or end-to-end co-training. We investigate whether world-model augmentation can be retrofitted around a frozen computer-use agent using only supervised fine-tuning. We introduce CUA-WM, a five-step pipeline that wraps OpenCUA-7B with state estimation via prompted L3 reasoning, candidate action generation, transition prediction via a LoRA adapter trained on 42,103 AgentNet examples, LLM-as-judge scoring, and response assembly. On OSWorld, CUA-WM improves overall success rate from 24.8% to 31.0% over vanilla OpenCUA-7B, with the largest gains on tasks where the base policy struggles. The framework also improves infeasible task recognition from 44.4% to 85.7%. During development, we identify a coordinate grounding artifact caused by temperature sampling in executable code and propose decoupled generation (sampling for reasoning, greedy for code) as a general fix. Our results demonstrate that SFT-based world modeling can meaningfully improve screenshot-based CUA performance without policy retraining.

1 Introduction

World-model reasoning, where an agent estimates the current state of its environment, predicts the outcomes of candidate actions, and selects the action with the best predicted outcome, has emerged as a powerful paradigm for improving agent performance. Wang et al. [13] formalize this as a POMDP decomposition into state estimation and transition modeling, showing that explicit world-model reasoning enables a 3B VLM to surpass proprietary models on diverse agent tasks. Chen et al. [4] demonstrate that confining world-model learning to a supervised fine-tuning stage yields further gains by avoiding multi-objective interference. In web navigation, Chae et al. [3] show that augmenting LLM agents with a transition-focused world model improves policy selection on WebArena and Mind2Web without retraining the policy, and Fang et al. [6] use a co-evolving world model as both a data generator and an imagination engine to sustain agent self-improvement. These results span web, GUI, and game environments, establishing world-model augmentation as a broadly applicable technique.

A common thread in these systems is that the world model is either trained jointly with the policy via reinforcement learning [13] or integrated into an end-to-end self-improvement loop [6]. A natural practical question arises: can world-model reasoning be retrofitted around an existing, capable computer-use agent (CUA) *post hoc*, using only supervised fine-tuning for the transition component, without modifying the base policy or requiring RL infrastructure? This is a relevant question for practitioners who have access to a frozen CUA and limited compute, and want to know whether wrapping it in a world-model framework improves task performance.

Preprint.

We investigate this question by building CUA-WM, a modular world-model framework around OpenCUA-7B [14], an open-source CUA built on Qwen2.5-VL [2]. CUA-WM implements a five-step pipeline executed at each agent step: (1) state estimation, which repurposes OpenCUA's L3 reasoning format to extract a natural-language description of the current screen state; (2) candidate generation, which produces N alternative action proposals using the L2 reasoning format; (3) transition prediction, which uses a LoRA-fine-tuned adapter to predict the expected screen changes for each candidate; (4) scoring, which uses the base model as an LLM-as-judge to rate each predicted transition against the task goal; and (5) response assembly, which formats the winning candidate's output for the evaluation environment. The transition adapter is trained on 42,103 examples extracted from the AgentNet dataset [14] using TRL's SFTTrainer [12]. CUA-WM serves as a drop-in replacement for vanilla OpenCUA, maintaining full compatibility with the OSWorld benchmark [15].

On OSWorld, CUA-WM improves overall success rate from 24.8% to 31.0% over vanilla OpenCUA-7B, with gains concentrating in task categories where the base policy performs poorly. The framework is particularly effective on infeasible tasks (tasks that cannot be completed and test whether the agent terminates gracefully) where success rate increases from 44.4% to 85.7%. During development, we discovered that naive temperature sampling over PyAutoGUI code perturbs pixel coordinates and degrades performance; a decoupled generation strategy that applies sampling only to reasoning sections and greedy decoding to code resolves this issue. These results demonstrate that SFT-based world modeling can meaningfully improve CUA task performance without policy retraining or RL, while also revealing concrete opportunities for further gains through stronger scoring mechanisms and RL-based optimization.

Our contributions are as follows:

1. CUA-WM, a five-step world-model framework for screenshot-based CUAs that can wrap any OpenCUA-compatible model without retraining the base policy.

2. A LoRA transition predictor trained on 42,103 AgentNet examples, achieving a validation loss of 0.577 with zero empty predictions across the full validation set.

3. Identification of a coordinate grounding artifact caused by temperature sampling in PyAutoGUI code generation, and a logits-processor-based fix (decoupled generation) that preserves reasoning diversity while maintaining coordinate precision.

2 Related Works

2.1 World Models

The concept of a world model, an internal representation that an agent uses to simulate and predict how its environment evolves in response to actions, has roots in cognitive science and control theory [7]. Ha and Schmidhuber formalized this idea for deep reinforcement learning, training a recurrent neural network to learn a compressed spatial and temporal representation of an environment, enabling policy optimization entirely within the learned model (i.e., "learning in imagination"). LeCun [9] later proposed a broader cognitive architecture in which a configurable predictive world model, realized as a hierarchical Joint-Embedding Predictive Architecture (JEPA), enables agents to plan by simulating candidate action sequences in an abstract representation space. Assran et al. [1] provided the first concrete instantiation of this vision with I-JEPA, which learns visual representations by predicting in latent space rather than reconstructing pixels.

A recent comprehensive survey by Ding et al. [5] organizes the growing literature along two primary functions: (1) constructing internal representations to understand the current state of the world, and (2) predicting future states to simulate outcomes and guide decision-making. Our work draws on both functions: state estimation (function 1) and transition prediction (function 2). Xing et al. [16] argue that contemporary world models place excessive emphasis on video generation, and that a world model should instead serve as a "sandbox for reasoning and thought experiment." This perspective aligns with our approach, which represents states and transitions as natural language rather than pixel-level predictions, using the world model to reason about action consequences rather than to generate visual simulations.

2.2 World Models for Agents

Chae et al. [3] introduced WMA-Agents, which augment LLM-based web agents with a world model trained to predict the outcomes of actions in web environments. To make prediction tractable over long HTML observations, they propose *transition-focused observation abstraction*, where the world model outputs free-form natural language descriptions of state differences rather than full next-state observations. The world model is used at inference time to simulate and score candidate actions, improving policy selection on WebArena and Mind2Web without additional policy training.

Wang et al. [13] formalize VLM agent tasks as a Partially Observable Markov Decision Process (POMDP) and decompose the agent's reasoning into two components: state estimation $P(\hat{s}_t \mid o_t)$ and transition modeling $P(\hat{s}_{t+1} \mid o_t, \hat{s}_t, \hat{a}_t)$. Their VAGEN framework trains both components via reinforcement learning, rewarding accurate state predictions alongside task completion. They find that this decomposition is critical for success and that natural language state representations excel at capturing semantic relationships for general tasks. A 3B model trained with VAGEN achieves strong performance across diverse, yet simplistic, agent tasks, surpassing several proprietary models.

Chen et al. [4] build on a similar state estimation and transition decomposition but confine world-model learning to a supervised fine-tuning (SFT) stage, reserving RL training for the accuracy objective alone. They observe that this separation avoids the multi-objective interference that can arise in VAGEN's joint training, yielding substantial gains on their testbed. This finding is particularly relevant to our work, as we also adopt an SFT-based approach to transition modeling.

Fang et al. [6] propose WebEvolver, a self-improvement framework that co-trains a world model alongside a web agent. The world model serves dual roles, namely, as (1) a virtual web server generating synthetic training trajectories, and (2) an imagination engine for look-ahead planning during inference. By scoring candidate actions through simulated multi-step rollouts, the world model enables continued agent improvement beyond the stagnation point typically observed in autonomous learning cycles.

Liu et al. [10] extend multi-agent reinforcement learning to LLM collaboration with CoMLRL, providing implementations of algorithms such as MAGRPO for training multiple LLMs to cooperate. While their focus is on multi-agent coordination rather than world modeling per se, their GRPO-based training framework represents a natural next step for optimizing world-model-augmented policies.

2.3 Computer-Use Agents

Computer-use agents (CUAs) operate directly from screenshots, accessibility trees, or raw HTML of desktop environments, producing executable actions such as mouse clicks, keyboard inputs, and hotkey combinations. Wang et al. [14] introduced OpenCUA, an open-source CUA built on Qwen2.5-VL [2] with a three-level chain-of-thought reasoning hierarchy: L1 (executable code), L2 (reflective planning with progress assessment and action analysis), and L3 (contextual observation of the screen state followed by L2-style reasoning). OpenCUA outputs actions as PyAutoGUI code containing pixel coordinates (e.g., `pyautogui.click(x=742, y=356)`), which are generated autoregressively as sequences of digit tokens.

Xie et al. [15] introduced OSWorld, a benchmark for evaluating multimodal agents on open-ended tasks in real computer environments spanning multiple applications and operating systems. OSWorld provides a scalable evaluation infrastructure with execution-based task verification, and has become a standard benchmark for CUA development. We evaluate our framework on OSWorld using OpenCUA-7B as the base policy.

2.4 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [8] enables parameter-efficient fine-tuning by injecting trainable low-rank decomposition matrices into frozen pretrained weights, allowing task-specific adaptation at a fraction of the full fine-tuning cost. LoRA has become a standard approach for adapting large language models to downstream tasks, and has been extended to vision-language models for domain-specific capabilities. We adopt LoRA to train a transition prediction adapter on trajectory data from the AgentNet dataset [14], using TRL's SFTTrainer [12] for supervised fine-tuning.

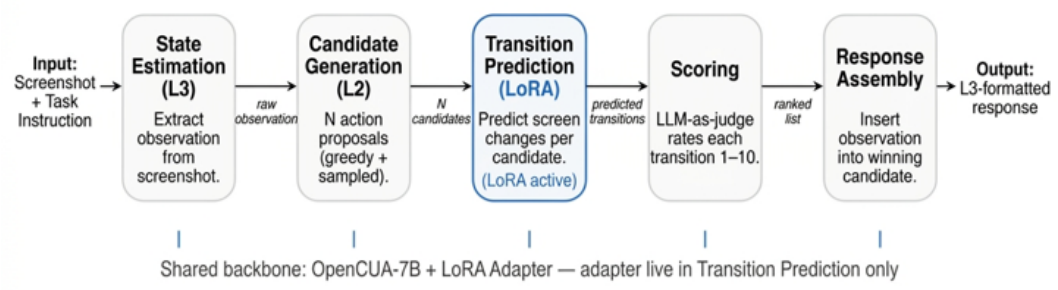

Figure 1: Pipeline overview. For each OSWorld step, the framework receives a screenshot and task instruction, then executes five stages: (1) state estimation via the L3 reasoning format, (2) candidate action generation via L2 reasoning, (3) transition prediction using a LoRA adapter, (4) scoring via LLM-as-judge, and (5) response assembly into L3 format. A single model serves all roles; the LoRA adapter is enabled only during Step 3 and disabled for all other steps.

3 Methodology

3.1 Overview

We construct a five-step pipeline that wraps around a frozen OpenCUA-7B model, augmenting it with world-model reasoning while maintaining full compatibility with the OSWorld evaluation interface. The pipeline serves as a drop-in replacement for vanilla OpenCUA since it accepts the same request format (screenshot + task instruction) and returns the same response format (chain-of-thought reasoning + PyAutoGUI code). Figure 1 illustrates the pipeline architecture.

The framework uses a single OpenCUA-7B model instance loaded with a LoRA adapter via PEFT's `PeftModel` interface [11]. The adapter is dynamically toggled per step. That is, enabled for transition prediction (Step 3) and disabled via a context manager for all other generation calls (Steps 1, 2, 4). This design avoids loading multiple model copies and allows the same model to serve as both the base policy and the world model.

Following the state estimation and transition modeling decomposition proposed by Wang et al. [13] and Chen et al. [4], our pipeline separates *what is the current state?* (Step 1) from *what will happen next?* (Step 3). Unlike VAGEN, which trains both components via RL, and SPA, which trains the world model via SFT and the policy via RL, we use SFT only due to resource constraints for the transition model and leave the base policy frozen. This isolates the contribution of the transition predictor from the effects of policy optimization.

3.2 State Estimation

The state estimation step extracts a natural-language description of the current screen state from the screenshot. Following VAGEN's formulation, this corresponds to computing an approximate belief state $\hat{s}_t$ from the observation o_t: $P(\hat{s}_t \mid o_t)$.

Rather than training a separate state estimation module, we repurpose OpenCUA's existing L3 reasoning capability. OpenCUA's L3 reasoning format begins with an Observation section that describes the screen state in detail before proceeding to Thought, Action, and Code sections. The Observation section includes the active application, visible UI elements, text content, and any elements relevant to the task. We exploit this by prompting the model with the L3 system prompt and truncating generation before the Thought section begins, extracting only the observation.

Concretely, we swap the incoming L2 system prompt (provided by OSWorld) for the L3 system prompt and generate with greedy decoding ($\tau = 0$), using early stopping at section headers (e.g., "Thought:", "Action:", "Code:"). The generated text is parsed to extract the Observation section via pattern matching. If no valid observation is found, the pipeline proceeds with an empty observation and the final response falls back to the raw L2 output.

This approach avoids training a separate state estimator and leverages the base model's existing screen comprehension abilities. It aligns with the finding from Wang et al. [13] that natural language state representations are effective at capturing semantic relationships for general agent tasks.

3.3 Candidate Generation

The candidate generation step produces N alternative action proposals using OpenCUA's L2 reasoning format, which is the same format used by vanilla OpenCUA and the best performing out of L1, L2, and L3 reasoning [14]. Following the approach of Chae et al. [3], who generate multiple candidate actions for the world model to evaluate, we sample N candidates to provide diversity for the subsequent scoring stage.

The first candidate is generated with greedy decoding ($\tau = 0$), matching vanilla OpenCUA's behavior exactly. Subsequent candidates use temperature sampling ($\tau = 0.7$ by default) to explore alternative reasoning paths that may lead to different actions. All candidates use the original L2 system prompt and the current screenshot, with the LoRA adapter disabled.

Decoupled generation. OpenCUA outputs actions as PyAutoGUI code containing pixel coordinates (e.g., `pyautogui.click(x=742, y=356)`), generated autoregressively as sequences of digit tokens. Temperature sampling, while beneficial for reasoning diversity, perturbs the probability distribution over these digit tokens, assigning non-negligible probability mass to neighboring values and potentially shifting coordinates by enough pixels to target incorrect UI elements.

To address this, we introduce *decoupled generation*, where temperature sampling is applied during the Thought and Action sections of the output (where reasoning diversity is beneficial), but the model switches to greedy decoding once the Code section begins (where coordinate precision is essential). This is implemented as a custom `LogitsProcessor` that monitors the generated text for Code section markers (e.g., `"Code:"`). Upon detecting a marker, the processor forces argmax token selection for all subsequent tokens by zeroing out all logits except the maximum. This requires no changes to the model or the generation API since the processor is attached as a callback to a standard `generate()` call and operates within a single forward pass.

3.4 Transition Prediction

The transition prediction step constitutes the world model component of our framework. For each candidate action, the LoRA adapter is enabled and the model receives the current state description (from Step 1) along with the candidate's action text, and predicts the expected changes to the screen state. Following VAGEN's notation, this corresponds to the transition model $P(\hat{s}_{t+1} \mid o_t, \hat{s}_t, \hat{a}_t)$.

The transition model is text-only, as it receives and produces natural language, with no image input. This follows the transition-focused observation abstraction approach of Chae et al. [3], where predictions are free-form descriptions of state changes rather than full next-state observations or pixel-level reconstructions. As argued by Xing et al. [16], this treats the world model as a reasoning sandbox rather than a visual simulator.

Training data. We derive training examples from the AgentNet Ubuntu 5K dataset [14], which contains 5,000 annotated desktop task trajectories totaling 82,448 steps. Each step includes an observation (L3-style state description), an action (descriptive text), executable code, and a reflection describing what changed after the action was executed. We extract transition pairs from steps that satisfy the following criteria: (1) non-empty reflection (providing ground-truth transition descriptions), (2) non-empty observation and action, (3) not marked as incorrect (`last_step_correct` $\neq$ `False`), (4) not marked as redundant, (5) not a terminate action, and (6) not the final step in the trajectory (ensuring a next state exists). This yields 42,103 examples, split 90/10 into 37,893 training and 4,210 validation examples.

Each training example is formatted as a three-turn chat conversation. That is, a system message defining the world model's role (predicting screen state changes given a current state and planned action), a user message containing the current observation and action, and an assistant message containing the ground-truth reflection. This format is compatible with standard chat-template-based SFT.

Training configuration. The adapter is trained using LoRA [8] with rank $r = 16$, scaling factor $\alpha = 32$, and dropout 0.05, targeting the query, key, value, and output projection layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`) of the base model's attention blocks. We use TRL's SFTTrainer [12] with a cosine learning rate schedule (peak 2×10^{-4}), warmup ratio 0.05, weight decay 0.01, effective batch size 16 (per-device batch size 2 $\times$ gradient accumulation 8), maximum sequence length 2,048, and bfloat16 precision with gradient checkpointing. Training runs for 3 epochs. The resulting adapter is 39MB.

3.5 Scoring and Selection

The scoring step evaluates each candidate by rating how well its predicted transition advances toward the task goal. The base model (with LoRA adapter disabled) acts as an LLM-as-judge, following the pattern used by Fang et al. [6] for evaluating imagined trajectories. For each candidate, the model receives a text-only prompt containing the task instruction, the candidate's action, and the predicted transition, and is asked to respond with a single integer from 1 to 10.

Candidates are sorted by score in descending order. Ties are broken by preferring lower candidate indices (i.e., the greedy candidate at index 0 is preferred over sampled candidates), reflecting the assumption that the deterministic candidate is a safer default when the judge cannot differentiate.

When only a single candidate is generated ($N = 1$), the world model and scoring steps are skipped entirely, and the pipeline reduces to state estimation followed by vanilla L2 generation with observation prepended. That is, equivalent to an L3-style response from vanilla OpenCUA.

3.6 Response Assembly

The final step constructs a response in OpenCUA's L3 format by inserting the state estimation observation into the winning candidate's L2 output. The L2 format follows the structure: Thought $\rightarrow$ Action $\rightarrow$ Code. The L3 format prepends an Observation section: Observation $\rightarrow$ Thought $\rightarrow$ Action $\rightarrow$ Code.

The assembly step locates the Thought section header in the winning candidate's raw output and inserts the observation text before it. If the observation is empty (i.e., state estimation failed to extract a valid description), the raw L2 output is returned unchanged. The resulting response is returned to OSWorld through the same API endpoint used by vanilla OpenCUA, ensuring transparent substitution.

4 Experimental Setup

4.1 Benchmark

We evaluate on OSWorld [15], a benchmark for multimodal agents operating in real desktop environments. OSWorld provides execution-based task verification. That is, each task specifies an initial VM state and a programmatic evaluation script that checks whether the task was completed successfully after the agent's interaction. We use the Ubuntu domain, which includes tasks spanning LibreOffice applications, file management, terminal operations, and system configuration. Each task is run with a maximum step limit of 30 actions. The primary metric is *success rate*: the fraction of tasks for which the evaluation script confirms correct completion.

We report results on the full OSWorld Ubuntu task set, as well as an initial 20-task subset used during development.

4.2 Configurations

We compare three configurations, all using OpenCUA-7B as the base model:

1. **Vanilla OpenCUA.** The unmodified OpenCUA-7B model with greedy decoding ($\tau = 0$), serving as the baseline. This matches the default inference configuration described by Wang et al. [14].

Table 1: Overall results on OSWorld. Success rate is the fraction of tasks completed successfully. "Feasible" and "Infeasible" refer to whether the task is possible for the agent to complete. CUA-WM uses $N = 2$ candidates with decoupled generation. Note that OSWorld occasionally fails to generate a score with no fault to the agent, causing the small difference in number of tasks.

Configuration	All Tasks	Feasible	Infeasible
Vanilla OpenCUA-7B	24.8% (89/359)	23.2% (77/332)	44.4% (12/27)
CUA-WM	31.0% (109/352)	26.2% (85/324)	85.7% (24/28)
Δ	+6.2 pp	+3.0 pp	+41.3 pp

2. **CUA-WM v1.** The world-model-augmented pipeline (Section 3) with $N = 2$ candidates. The first candidate uses greedy decoding; the second uses temperature sampling ($\tau = 0.7$) applied uniformly across all output sections (Thought, Action, and Code).

3. **CUA-WM v2.** Identical to v1, but with decoupled generation (cf. Section 3.3): temperature sampling is applied only during the Thought and Action sections, and the model switches to greedy decoding once the Code section begins. This is the version used for the full benchmark evaluation.

The comparison between v1 and v2 isolates the effect of temperature sampling on the Code section, while the comparison between v2 and vanilla isolates the contribution of the world-model pipeline itself.

4.3 World Model Training

The LoRA transition adapter is trained on 37,893 examples derived from the AgentNet Ubuntu 5K dataset, with 4,210 held out for validation (cf. Section 3.4). Training runs for 3 epochs (7,107 steps) on a single NVIDIA L40 GPU (48GB VRAM) and completes in approximately 11.7 hours. The final validation loss is 0.577, decreasing steadily from an initial loss of 0.737 with no signs of overfitting. Evaluation of the adapter on all 4,210 validation examples yields zero empty predictions, with an average inference time of 7.4 seconds per example.

4.4 Compute

All model inference is performed on a single GPU with a minimum of 40GB VRAM (typically NVIDIA L40 or L40S). The OpenCUA-7B model with the LoRA adapter loaded requires approximately 14GB VRAM in bfloat16 precision. Per-step latency for the full pipeline with $N = 2$ candidates is approximately 48 seconds, compared to roughly 10 seconds for vanilla OpenCUA. The additional latency arises from the state estimation call (Step 1), the second candidate generation (Step 2), two text-only transition predictions (Step 3), and two text-only scoring calls (Step 4). Steps 3 and 4 are text-only and thus substantially faster than the vision-language calls in Steps 1 and 2.

5 Results

5.1 Main Results

Table 1 presents the overall results on OSWorld. CUA-WM achieves a success rate of 31.0% across all tasks, compared to 24.8% for vanilla OpenCUA, which is a 6.2 percentage point improvement (25% relative gain). On feasible tasks, CUA-WM improves from 23.2% to 26.2%. On infeasible tasks (that is, tasks that cannot be completed and are included to test whether the agent terminates gracefully) CUA-WM achieves 85.7% compared to vanilla's 44.4%.

The total number of tasks differs slightly between configurations (359 vs. 352) due to a small number of tasks that failed to initialize or timed out during environment setup in each run. These failures are independent of the agent configuration.

Table 2: Per-category success rates on feasible tasks. Δ is the percentage point difference (CUA-WM $-$ vanilla). Categories are sorted by the number of feasible tasks. Note that OSWorld occasionally fails to generate a score with no fault to the agent, causing the small difference in number of tasks.

Category	Vanilla		CUA-WM		Δ (pp)
	Tasks	Rate	Tasks	Rate	
Multi Apps	92	8.7%	84	13.1%	+4.4
LibreOffice Impress	47	31.9%	47	27.7%	−4.3
LibreOffice Calc	46	2.2%	46	8.7%	+6.5
Chrome	43	30.2%	44	38.6%	+8.4
LibreOffice Writer	22	31.8%	22	36.4%	+4.5
OS	19	31.6%	18	33.3%	+1.8
VS Code	18	55.6%	18	44.4%	−11.1
GIMP	16	37.5%	16	43.8%	+6.3
VLC	15	20.0%	15	33.3%	+13.3
Thunderbird	14	57.1%	14	42.9%	−14.3
All Feasible	332	23.2%	324	26.2%	+3.0

5.2 Per-Category Breakdown

Table 2 reports success rates by application category on feasible tasks. CUA-WM improves over vanilla OpenCUA in seven of ten categories, with the largest gains in GIMP (+6.3 pp), Chrome (+8.4 pp), VLC (+13.3 pp), LibreOffice Calc (+6.5 pp), and Multi Apps (+4.4 pp). Three categories show regressions: LibreOffice Impress (-4.3 pp), Thunderbird (-14.3 pp), and VS Code (-11.1 pp).

The regressions tend to occur in categories where vanilla OpenCUA already performs well (Thunderbird: 57.1%, VS Code: 55.6%), while the gains tend to occur in categories where vanilla performs poorly (LibreOffice Calc: 2.2%, Multi Apps: 8.7%, VLC: 20.0%). This pattern suggests that the world model may be most beneficial when the base policy struggles, but may introduce unnecessary overhead or occasional misselection in tasks the base policy already handles reliably.

5.3 Infeasible Task Handling

CUA-WM's improvement on infeasible tasks (44.4% $\rightarrow$ 85.7%) is substantially larger than its improvement on feasible tasks and was not an explicit design goal. We hypothesize that the world-model pipeline provides two mechanisms that help the agent recognize infeasible tasks. First, the state estimation step (Step 1) produces an explicit description of the current screen state, making the absence of required elements more salient. Second, the transition prediction step (Step 3) may predict that a candidate action will have no meaningful effect, a signal which the agent can use to determine that progress toward the goal is not possible. Together, these steps create an interpretive layer between observation and action that facilitates graceful termination.

5.4 Decoupled Generation

During initial development, we evaluated an earlier version of CUA-WM that applied temperature sampling uniformly across the full output, including the Code section containing PyAutoGUI coordinates. On a 20-task pilot, this earlier version achieved a success rate of approximately 8.3%, compared to 17.3% for vanilla. Analysis revealed that temperature sampling perturbed digit tokens in coordinate values (e.g., shifting `x=742` to `x=748`), causing clicks to land on incorrect UI elements.

The decoupled generation mechanism (Section 3.3) resolves this by restricting temperature sampling to the Thought and Action sections while using greedy decoding for the Code section. Smoke tests confirm that both greedy and sampled candidates produce identical PyAutoGUI code despite differing reasoning traces. The full-benchmark results reported in Tables 1–3 use this new configuration.

Table 3: Per-category success rates on infeasible tasks. Many categories contain very few infeasible tasks, limiting per-category interpretation. LibreOffice Impress has no infeasible tasks in either run. Note that OSWorld occasionally fails to generate a score with no fault to the agent, causing the small difference in number of tasks.

Category	Vanilla		CUA-WM	
	Tasks	Succeeded	Tasks	Succeeded
GIMP	10	4	10	9
VS Code	5	3	5	4
OS	4	0	5	5
Chrome	2	2	2	2
VLC	2	1	2	1
LibreOffice Calc	1	0	1	0
LibreOffice Writer	1	0	1	1
Multi Apps	1	1	1	1
Thunderbird	1	1	1	1
All Infeasible	27	12 (44.4%)	28	24 (85.7%)

6 Discussion

SFT-based world modeling without policy modification. CUA-WM demonstrates that a world-model framework can be retrofitted around a frozen CUA using only a supervised LoRA adapter, without modifying the base policy or requiring reinforcement learning. The overall improvement from 24.8% to 31.0% suggests that the transition predictions provide useful signal for action selection. This is notable because it contrasts with the approach taken by VAGEN [13] and SPA [4], which train the policy jointly with or after the world model via RL. Our results suggest that even without RL-based policy optimization, the SFT-trained transition model contributes to improved task performance, although the magnitude of improvement leaves open the question of how much further RL-based training could push these gains.

Where does world modeling help? The per-category results reveal a suggestive pattern, namely that CUA-WM's gains concentrate in categories where vanilla OpenCUA performs poorly (LibreOffice Calc: 2.2% $\rightarrow$ 8.7%, Multi Apps: 8.7% $\rightarrow$ 13.1%, VLC: 20.0% $\rightarrow$ 33.3%), while regressions occur in categories where vanilla already performs well (Thunderbird: 57.1% $\rightarrow$ 42.9%, VS Code: 55.6% $\rightarrow$ 44.4%). One interpretation is that the world model provides the most value when the base policy is uncertain, whereas for tasks the base policy already handles reliably, the additional pipeline steps introduce opportunities for misselection without commensurate benefit. However, the per-category task set sizes are small enough that this pattern may not generalize, and we cannot rule out confounding factors such as differences in task complexity or interface structure across categories.

Infeasible task recognition. The most striking result is CUA-WM's improvement on infeasible tasks (44.4% $\rightarrow$ 85.7%), which was not an explicit design objective. This suggests that the world-model pipeline creates a structured reasoning layer that aids not only in action selection but also in recognizing when no productive action is available. The state estimation step makes the current screen state explicit in natural language, and the transition prediction step surfaces predictions such as "this action will have no visible effect", both of which may help the agent decide to terminate rather than perseverate. This finding has practical implications: in real-world deployments, an agent's ability to recognize and gracefully exit impossible tasks is arguably as important as its ability to complete feasible ones.

Decoupled generation as a general consideration. The failure of the earlier version of CUA-WM (8.3% on a 20-task pilot) and subsequent recovery via decoupled generation highlight a practical challenge for any framework that samples candidate actions from a CUA producing executable code with numerical parameters. Temperature sampling over digit tokens is inherently imprecise for values where small perturbations have large semantic consequences since a coordinate shift of a few pixels can target an entirely different UI element. Decoupled generation, which applies temperature sampling only to natural-language reasoning and switches to greedy decoding for code,

is a lightweight solution that requires no model modification. We note that this issue is specific to CUAs that embed coordinates directly in their output format; architectures that separate action description from grounding (e.g., outputting "click the OK button" which a separate module resolves to coordinates) would not face this problem.

Path forward. Several directions could strengthen CUA-WM. First, replacing the LLM-as-judge scoring with RL-based policy optimization (for example, using GRPO as in CoMLRL [10]) could enable the model to learn from the transition predictions rather than relying on a single-call rating. Second, pairwise comparison ("which transition better advances the goal?") may be more reliable than independent 1-10 scoring, particularly for a 7B model serving as its own judge. Third, scaling to larger base models (e.g., OpenCUA-72B) would test whether the gains from world modeling compound with stronger base policies or exhibit diminishing returns. Finally, the transition model could be used for multi-step lookahead (that is, simulating several steps ahead via chained transition predictions) rather than the single-step prediction used in our current pipeline.

7 Limitations

Our evaluation is based on a single run of each configuration on OSWorld, without repeated trials. As a result, we cannot report confidence intervals or statistical significance for the observed differences. The overall improvement (24.8% $\rightarrow$ 31.0%) is based on approximately 350 tasks, and the per-category breakdowns involve substantially fewer tasks per category, making individual category-level comparisons unreliable.

We evaluate only one base model size (OpenCUA-7B). It is unknown whether the gains from world-model augmentation would increase, persist, or diminish with larger models such as OpenCUA-72B, which may already internalize the reasoning that the world model provides.

The transition adapter is trained exclusively on AgentNet Ubuntu 5K trajectories. While our evaluation is also on Ubuntu tasks (via OSWorld), the adapter may not generalize to other operating systems or application domains not represented in the training data.

CUA-WM uses the same 7B model as both the base policy and the LLM-as-judge scorer. This creates a ceiling on scoring quality: the judge may lack the capacity to reliably differentiate candidate transitions, particularly when candidates are similar. A stronger external judge or a dedicated scoring model could yield different results.

We do not ablate the individual pipeline steps. The current evaluation compares the full pipeline against vanilla OpenCUA, but does not isolate the marginal contribution of state estimation, transition prediction, or scoring independently. For example, it is possible that the state estimation step alone (producing L3-style responses) accounts for most of the improvement, and that the transition prediction and scoring steps add little.

Finally, our framework uses supervised fine-tuning only. We do not evaluate RL-based approaches to training the transition model or optimizing the policy, as explored by VAGEN [13] and SPA [4], due to resource constraints. Our results therefore speak to the SFT-only regime and do not bound the potential of world-model augmentation for CUAs more broadly.

8 Conclusion

We introduced CUA-WM, a modular world-model framework that augments a frozen OpenCUA-7B agent with state estimation, transition prediction via a LoRA adapter, and LLM-as-judge action selection. On OSWorld, CUA-WM improves overall success rate from 24.8% to 31.0% without any modification to the base policy or reinforcement learning training. The framework is particularly effective on infeasible tasks, where success rate increases from 44.4% to 85.7%, and on task categories where the base policy struggles. We also identify and resolve a coordinate grounding artifact caused by temperature sampling in executable code generation, proposing decoupled generation as a lightweight, general-purpose fix for CUA systems that sample candidate actions. Our results demonstrate that SFT-based world modeling is a viable approach for augmenting screenshot-based computer-use agents, and that the transition model, scoring mechanism, and action space design each present concrete opportunities for further improvement.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, June 2023.

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report, 2025. URL https://arxiv.org/abs/2502.13923.

[3] Hyungjoo Chae, Namyoung Kim, Kai Tzu iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=moWiYJuSGF.

[4] Shiqi Chen, Tongyao Zhu, Zian Wang, Jinghan Zhang, Teng Xiao, Kangrui Wang, Siyang Gao, Yee Whye Teh, Junxian He, and Manling Li. Internalizing world models via self-play finetuning for agentic RL, 2026. URL https://openreview.net/forum?id=K8wCGMzeuY.

[5] Jingtao Ding, Yunke Zhang, Yu Shang, Yuheng Zhang, Zefang Zong, Jie Feng, Yuan Yuan, Hongyuan Su, Nian Li, Nicholas Sukiennik, Fengli Xu, and Yong Li. Understanding world or predicting future? a comprehensive survey of world models. *ACM Comput. Surv.*, 58(3), September 2025. ISSN 0360-0300. doi: 10.1145/3746449. URL https://doi.org/10.1145/3746449.

[6] Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. WebEvolver: Enhancing web agent self-improvement with co-evolving world model. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 8959–8975, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.454. URL https://aclanthology.org/2025.emnlp-main.454/.

[7] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/2de5d16682c3c35007e4e92982f1a2ba-Paper.pdf.

[8] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[9] Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.

[10] Shuo Liu, Zeyu Liang, Xueguang Lyu, and Christopher Amato. Llm collaboration with multi-agent reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 32150–32158, 2026.

[11] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

[12] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020. URL https://github.com/huggingface/trl.

[13] Kangrui Wang, Pingyue Zhang, Zihan Wang, Yaning Gao, Linjie Li, Qineng Wang, Hanyang Chen, Yiping Lu, Zhengyuan Yang, Lijuan Wang, Ranjay Krishna, Jiajun Wu, Li Fei-Fei, Yejin Choi, and Manling Li. VAGEN: Reinforcing world model reasoning for multi-turn VLM agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=xpjWEgf8zi.

[14] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Jiarui Hu, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Dikang Du, Hao Hu, Huarong Chen, Zaida Zhou, Haotian Yao, Ziwei Chen, Qizheng Gu, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Flood Sung, Y. Charles, Zhilin Yang, and Tao Yu. Opencua: Open foundations for computer-use agents, 2025. URL https://arxiv.org/abs/2508.09123.

[15] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024.

[16] Eric Xing, Mingkai Deng, Jinyu Hou, and Zhiting Hu. Critiques of world models, 2025. URL https://arxiv.org/abs/2507.05169.